

Сервис прогнозирования отказов датчиков инженерных коллекторов

Сопроводительная документация к решению

ЛЦТ-2026, задача 8, АО «Москоллектор». Команда «Скайнет». Состояние на 28.09.2026.

Документ закрывает требование ТЗ разд. 14 о сопроводительной документации: он описывает методы обработки данных (разд. 3), условия и ограничения решения (разд. 5), функциональную и компонентную архитектуру (разд. 6 и 7), сборку и установку (разд. 9). Для проверки решения экспертами добавлены качество прогноза и способ его повторить (разд. 4), REST API (разд. 8), стек и лицензии (разд. 10).

Оглавление

1. Назначение и состав сдачи
2. Что прогнозирует сервис
3. Методы обработки данных
4. Качество прогноза и производительность
5. Условия и ограничения
6. Функциональная архитектура
7. Компонентная архитектура
8. REST API
9. Сборка и установка
10. Стек и лицензии
11. Как проверить решение за 15 минут

1. Назначение и состав сдачи

АО «Москоллектор» эксплуатирует 825 км коллекторов, в которых стоят датчики охранной и пожарной сигнализации, газового контроля и диспетчерского управления. В выгрузке, которую заказчик передал на хакатон, таких датчиков 11 485. Сервис каждые четыре минуты оценивает для каждого датчика риск отказа в ближайшие 24 часа, показывает диспетчеру, где на трассе коллектора этот риск сосредоточен, и сам заводит превентивную заявку на обслуживание, когда риск высокий.

Что дано	Где
Работающий стенд	https://moskollektor.mbogatyreva.ru — демо-учётки четырёх ролей показаны на экране входа, кнопка «Войти как»
Описание REST API (Swagger)	https://moskollektor.mbogatyreva.ru/docs , машинное описание — https://moskollektor.mbogatyreva.ru/openapi.json
Исходный код, миграции, развёртывание	https://github.com/miroslavabogatyreva/moskollektor
Все 175 пунктов ТЗ с отметкой о выполнении	файл <code>tz-requirements.md</code> в корне репозитория
Расчёт правил прогноза и его метрики	каталог <code>ml-model/sensor/</code> репозитория
Эта документация	<code>documentation.pdf</code> в корне репозитория, рядом с <code>README.md</code>

На стенде загружена полная выгрузка заказчика — 313 546 016 строк журнала СМБУ за 2019 год — июнь 2026 года, база занимает около 55 ГБ.

2. Что прогнозирует сервис

Отказ — это потеря связи с каналом датчика, а не поломка оборудования. Журнал СМВУ пишет оборванный шлейф, обесточенную линию и сгоревший датчик одним значением «Неисправен». Заказчик письменно подтвердил, что это «только факт потери связи с устройством, причину может выявить только выезд дежурной бригады на место». Актыв выездов и журнала ложных срабатываний заказчик не передаёт, поэтому отличить поломку датчика от обрыва линии по данным нельзя, и сервис прогнозирует то, что в данных есть.

Отказом считается эпизод, а не отдельная запись. Эпизод — непрерывная серия записей канала со значением неисправности, которую закрывает первая запись с любым другим значением. Отказом эпизод становится, если длится строго больше часа (3 600 секунд). Кроме «Неисправен», неисправностью считаются ещё три значения журнала: «Батарея неисправна», «Много неисправных устройств» и «Не определено». Определение записано в файле `contracts/failure.v3.json`.

Эпизод, а не запись, выбран потому, что каналы «мигают». Например, канал 228571 «ТЕМП ПК275+9» за 2025 год записал «Норма» 22 167 раз и «Неисправен» 22 156 раз с медианой промежутка две секунды: по записям это было бы 22 тысячи отказов одного канала. Порог в один час выбрали мы: заказчик написал, что «разделение отказов, ложных срабатываний и отсутствия данных — задача участников».

Горизонт прогноза — 24 часа. Это целевой горизонт постановки. Вероятность на экране означает долю отказов канала в ближайшие 24 часа после момента расчёта.

3. Методы обработки данных

3.1. Исходные данные

Единственный источник фактов — обезличенная выгрузка СМВУ. Заказчик назвал её полной и окончательной: «Догрузка данных не планируется».

Файл выгрузки	Что в нём	Куда ложится	Строк
справочник_объектов_диспетчер.csv	дерево объектов диспетчера	smvu.object_tree	95
справочник_каналов_датчиков.csv	каналы датчиков	smvu.channel	11 485
ext-journal-2019.csv ... ext-journal-2026.csv , восемь файлов	журнал СМВУ	smvu.reading	313 546 016 в файлах, 312 982 076 в базе

Разница между файлами и базой — 563 940 строк. Это точные копии строк в двух окнах, 05.07.2019 и 24–29.11.2023; заливка отбросила их первичным ключом (journal_id, read_time).

СМВУ пишет в журнал не опрос датчика по расписанию, а смену состояния канала: «Норма», «Неисправен», «Выключен», числовое значение. Поэтому признак ухудшения в журнале несёт не само значение, а история смен.

3.2. Заливка и контроль качества

Выгрузку заливает модуль командной строки backend/app/ingest/smvu_csv.py. Порядок задают внешние ключи: сначала дерево объектов, потом справочник каналов, потом журнал. Журнал идёт командой COPY в помесечные партиции таблицы smvu.reading; годовые файлы можно лить в три потока, на полной выгрузке это около часа.

Строка, которую нельзя принять как есть, не останавливает заливку: она ложится в таблицу load.error с номером строки файла и причиной. Каждый запуск оставляет в load.batch отчёт о качестве — сколько строк принято и сколько отвергнуто. Тип датчика и инженерной системы сверяются со справочниками smvu.sensor_kind (19 типов) и smvu.system_kind (6 систем) с нормализацией пробелов, регистра и латинских букв, похожих на кириллические (backend/app/ingest/kind_names.py). Команда smvu_csv --check после заливки сверяет девять контрольных чисел: строки, каналы, участки, объекты, пустоту служебной партиции, месяцы без данных.

3.3. Ключ участка

Участка отдельной колонкой в выгрузке нет, сервис собирает его сам (backend/app/ingest/tag_to_section.py). Номер коллектора берётся из префикса тега до дефиса, пикет — из названия датчика. Например, тег 847-11.1.131.2. и название КД АВ ПК106 ур.3 дают участок 847:106. Пара нужна потому, что пикетаж у каждого коллектора свой и начинается с нуля. Так к участку привязываются 10 720 каналов из 11 485 (93,3 %), и получается 3 173 участка на 16 коллекторах верхнего уровня дерева диспетчера. Оставшиеся 765 каналов стоят в диспетчерских и охранных зонах без пикета: коллектор у них известен, участка нет.

3.4. Эпизоды отказа

Модуль `backend/app/ingest/model_failure_episodes.py` проходит журнал каждого канала в порядке `(read_time, journal_id)` и собирает эпизоды по определению из разд. 2 в таблицу `smvu.model_failure_episode`. Представление `smvu.model_failure_event` оставляет из них отказы – эпизоды длиннее часа и незакрытые на конец архива. На этом представлении стоят правила прогноза, карточка участка и замер метрик. Эталонная реализация того же алгоритма на Python лежит в том же модуле, и самопроверка сверяет с ней результат SQL на живых каналах.

3.5. Исключённые периоды и плановые работы

Что исключено	Почему	Как это сделано
Весь 2021 год, 53 698 948 записей (17,1 % архива)	заказчик рекомендовал письменно: «Всплеск тревог в 2021 году вызван переходом на новую версию системы мониторинга... Этот период рекомендуется исключить из анализа»	окно с причиной <code>vendor_migration</code> в <code>smvu.data_outage</code> (<code>db/migrations/040_data_outage_reason.sql</code>); таблицы баллов правил считаются на окне 01.04.2022 – 31.12.2025, 2021 год в него не входит
7–8 апреля 2024 года и 1 июня 2026 года	в выгрузке нет ни одной записи по всему парку, отличить «ничего не произошло» от «выгрузка не доехала» нельзя	окна <code>export_gap</code> в <code>smvu.data_outage</code> (<code>db/migrations/017_fault_episodes.sql</code>); эпизод поверх такого окна помечается флагом <code>spans_outage</code> , его длительность завышена пропуском
Плановый демонтаж газовых датчиков на поверку	снятый на поверку датчик пишет «Неисправен», но это не отказ	график ППР заказчика на 2026 год лежит в <code>maint.ppr_window</code> (сид <code>db/seed/ppr_2026.sql</code>); эпизоды внутри окон, сопоставленных с журналом уверенно, в правило не идут, а на экране подписаны «плановый демонтаж на поверку, не отказ»

График ППР обезличен своей нумерацией объектов, ключа к объектам выгрузки заказчик не дал. Мы сопоставили 26 строк графика с журналом по дате и числу датчиков: уверенно совпали две строки – «Объект 14», 33 датчика, 04.06.2026, и «Объект 9», 40 датчиков, 23.04.2026. Именно их окна исключаются.

3.6. Суточная свёртка

Читать 313 млн строк на каждый расчёт нельзя, поэтому журнал сворачивается по суткам: `feat.channel_daily` – по каналу, `feat.section_daily` – по участку. Функции `feat.refresh_channel_daily(d1, d2)` и `feat.refresh_section_daily(d1, d2)` пересчитывают окно в двое суток, потому что запись о вчерашнем событии иногда доезжает сегодня. По свёртке сервис определяет край данных: для выгрузки это 30.06.2026 23:59:59 по Москве.

3.7. Правила прогноза датчика

Прогноз продукта считают два правила. Пороги и баллы правил лежат в файле `backend/app/domain/sensor_rules.json`, код правил – функция `rule_split()` в `backend/app/domain/sensor_risk.py`. Файл правил получен скриптом `ml-model/sensor/rules.py` и воспроизводится байт в байт (разд. 4.6).

Правило давности работает на реальных отказах. Сервис смотрит, сколько часов назад начался последний подтверждённый отказ канала. Подтверждённый значит, что с начала эпизода прошёл час и одна секунда: раньше нельзя знать, что эпизод станет отказом. Если прошло меньше 24 часов, уровень датчика – «высокий риск», если меньше 72 часов – «наблюдать», иначе «норма».

Пороги 24 и 72 часа выбраны на январе–марте 2026 года из сетки 2, 6, 12, 24, 48, 72 и 168 часов. Порог «высокого риска» – по лучшей F1 (24 ч дали 0,043 против 0,040 у 12 ч и 0,035 у 48 ч), порог «наблюдать» – по лучшей F2 среди порогов больше 24 ч, потому что для наблюдения полнота важнее точности (72 ч дали 0,044 против 0,042 у 48 ч).

Балл датчика – частота, с которой канал с такой давностью отказывал в следующие 24 часа на обучающем окне 01.04.2022 – 31.12.2025. Окно выбора порогов и окно замера в обучающее не входят.

Давность последнего отказа	Пар «канал × сутки» в обучении	Балл (доля отказов за 24 ч)	Уровень
0–2 ч	23	0,304	высокий риск
2–6 ч	426	0,610	высокий риск
6–12 ч	2 203	0,142	высокий риск
12–24 ч	539	0,019	высокий риск
24–48 ч	2 611	0,075	наблюдать
48–72 ч	2 414	0,014	наблюдать
72–168 ч	9 437	0,006	норма
168–720 ч	51 046	0,002	норма
30 суток и больше или отказов не было	15 665 751	0,00014	норма

Балл внутри уровня не монотонен, и это свойство данных, а не ошибка: канал, отказавший 2–6 часов назад, в 61 % случаев отказывает снова в ближайшие сутки, а отказавший 12–24 часа назад – в 2 %. Поэтому уровень задаётся порогом давности, а не баллом, и датчики сортируются сначала по уровню, потом по баллу.

Правило предвестника работает только в режиме «с синтетикой» – на синтетических паспортах оборудования. Настоящих паспортов, проверок и предупреждений приборов заказчик не передаёт, поэтому мы их смоделировали (`backend/app/domain/failure_sim.py`): по паспорту датчика (вид оборудования, дата ввода, срок службы) и датам проверок генератор выбирает момент начала ухудшения по распределению Вейбулла. В этот момент появляется «предупреждение прибора», а через интервал P-F, равный двум суткам, – искусственный отказ. На каждое истинное предупреждение генератор добавляет одно ложное. Генератор не читает реальные отказы, иначе правило выучило бы подсмотренный ответ. Если после предупреждения назначенный отказ ещё не наступил и наступит в ближайшие 24 часа, уровень – «высокий риск», если позже – «наблюдать»; балл – 0,50.

Режим «с синтетикой» объединяет правила: балл равен $1 - (1 - \text{давность}) \times (1 - \text{предвестник})$, уровень – выше из двух. Экран показывает, какая часть балла синтетическая, и позволяет выключить синтетику переключателем на дашборде или параметром `synthetic=0` в API.

Пример. Канал 93566 отказал 06.04.2026 около 18:40. В 21:00 правило давности подняло его на «высокий риск»: с отказа прошло около 2,3 часа, балл 0,610. Следующий отказ канала начался 07.04.2026 в 07:30, через 10,5 часа после предупреждения.

3.8. Прогноз участка

Экраны схемы, карточки участка и журнала прогнозов работают с участками. Модуль `backend/app/worker/run_sensors.py` переводит датчики в участки: для каждого участка берётся самый рискованный датчик — сначала по уровню, потом по баллу. Вероятность участка — балл этого датчика, класс риска участка — «высокий», если датчик на уровне «высокий риск». Объяснение участка — главная причина этого датчика и до двух следующих датчиков с риском. Участок без датчиков тоже получает строку прогноза с нулём.

Пример из кода: на участке три датчика, у одного отказ был 30 часов назад — это уровень «наблюдать» с баллом 0,0747, два других в норме. Вероятность участка 0,0747, класс «норма», текст «<имя датчика>: правило давности (наблюдать): последний отказ канала 30 ч назад».

3.9. Превентивные заявки

Заявку заводит расчёт, а не диспетчер (`backend/app/domain/order_rules.py`, функция `план_по_датчикам()`). Правило такое.

- **Когда.** На участок, где самый рискованный датчик на уровне «высокий риск». Уровень «наблюдать» заявку не заводит.
- **Без повторов.** Ключ заявки — участок и московские сутки среза, поэтому на участок приходится не больше одной заявки в сутки. Открытая заявка со сроком позже среза глушит новую. Участок под открытым нарядом-допуском заявку не получает: там уже работают люди.
- **Вид работ и приоритет** зависят от класса критичности участка. Класс А — пожарная или газовая охрана, В — охранная подсистема и диспетчерский контроль, С — температура и диагностика. Нормативы реакции по Регламенту заказчика — 16, 48 и 72 часа.
- **Срок** — момент, когда система заметила риск, плюс меньшее из двух чисел: норматива реакции приоритета и потолка превентивности из настроек.
- **Обоснование** называет датчик и правило, например: «У датчика ФВР1 ПК1089 сработало правило высокого риска: правило давности (высокий риск): отказ 3 ч назад. Балл датчика по правилам за 24 ч — 0.610, класс участка В».

Из заявки открывается прогноз, по которому она заведена. Статусы заявок сервис раз в 10 минут забирает из эмулятора системы заявок (разд. 5).

3.10. Расписание расчёта

Процесс `worker` держит планировщик `APScheduler` с четырьмя заданиями (`backend/app/worker/scheduler.py`).

Задание	Как часто	Что делает
расчёт	каждые 4 минуты	прогон <code>backend/app/worker/run.py</code> : край данных, свёртка за двое суток, правила датчиков по всему парку, прогноз участков, запись и рассылка уведомления, автозаявки; затем баллы всех датчиков в <code>pred.sensor_risk</code>
свёртка	каждые 5 минут	<code>feat.refresh_section_daily</code> за двое суток
погода	каждую минуту проверяет, пора ли	забирает час погоды в <code>ext.weather_hourly</code> ; в прогноз погода не входит
статусы заявок	каждые 10 минут	сверяет статусы заявок с системой заявок

Расчёт защищён блокировкой `pg_try_advisory_lock`: второй экземпляр `worker` не начнёт тот же расчёт, а сразу выйдет. Если прогон упал, прошлый прогноз остаётся на экране с пометкой «расчёт не завершён». Каждый прогон пишет в `pred.run` время шести стадий в миллисекундах.

4. Качество прогноза и производительность

4.1. Протокол замера

- Пороги правил выбраны на январе–марте 2026 года, замер сделан один раз на апреле–июне 2026 года. Баллы посчитаны на 01.04.2022 – 31.12.2025. Окна не пересекаются.
- Срез – каждые сутки в 21:00 по Москве, 90 срезов. На каждом срезе правило решает по каждому из 11 485 активных каналов, поднимать ли тревогу.
- Попадание – тревога, после которой отказ этого канала начался в пределах 24 часов. Счёт делает функция `evaluate_alerts()` из `code/predictive_metrics.py`.
- За апрель–июнь 2026 года в журнале 606 реальных отказов на 300 каналах.

4.2. Результаты

Режим	Попадания (TP)	Ложные тревоги (FP)	Пропуски (FN)	Precision	Recall
на реальных отказах, правило давности	59	434	547	0,120	0,097
с симуляцией: 606 реальных и 313 симулированных отказов	372	742	547	0,334	0,405

Precision 0,120 – это 59 попаданий из 493 тревог. Recall 0,097 – это 59 пойманных отказов из 606. 95-процентный интервал Уилсона для Precision – от 0,094 до 0,151, для Recall – от 0,076 до 0,124: верхние границы далеко от 0,7 и 0,5, так что дело не в малой выборке.

Строка «с симуляцией» – результат на смеси реальных и смоделированных отказов, а не качество на реальных данных. Она показывает, как сработает правило предвестника, если у заказчика появятся настоящие предупреждения приборов.

Сравнение со случайным выбором. Из 1 033 650 пар «канал × срез» (11 485 каналов на 90 срезов) отказ в следующие 24 часа идёт за долей 0,000475 пар. Тревога, поднятая наугад, попадала бы с такой же вероятностью. Правило давности попадает с вероятностью 0,120, то есть в $0,120 / 0,000475 \approx 252$ раза чаще случайного выбора.

4.3. Почему горизонт 24 часа

То же правило давности проверено на пяти горизонтах; числа – в файле `ml-model/sensor/horizons.json`, скрипт – `ml-model/sensor/horizons.py`. Precision растёт с горизонтом, но в основном за счёт самого окна: за 14 суток отказ случается почти в 30 раз чаще, чем за 12 часов, и угадать его проще даже наугад. Поэтому последний столбец делит Precision на долю пар «канал × срез», за которыми в пределах горизонта идёт отказ.

Горизонт	Precision	Recall	Во сколько раз лучше случайного выбора
12 часов	0,022	0,018	115
24 часа	0,120	0,097	252
3 суток	0,146	0,117	111
7 суток	0,199	0,158	67
14 суток	0,335	0,264	59

24 часа — целевой горизонт постановки, и на нём правило сильнее всего опережает случайный выбор.

4.4. Почему пороги 0,7 и 0,5 не достигнуты

Постановка называет плановые значения Precision больше 0,7 и Recall больше 0,5. Заказчик 19.09.2026 письменно пояснил, что это не жёсткие требования: «Указанные значения Precision 0.7 и Recall 0.5 являются плановыми (по умолчанию), а не жесткими требованиями. Если данные не позволяют достичь этих уровней, их можно снизить, обязательно обосновав это в описании MVP». Наше обоснование состоит из трёх фактов.

1. **Отказ канала редок.** Доля пар «канал × сутки», за которыми следует отказ, — 0,000475, то есть примерно одна на 2 100. Даже хорошее правило при такой базовой частоте даёт много ложных тревог на одно попадание.
2. **Предвестника отказа в журнале нет.** Журнал пишет смены состояния канала, а не показания, по которым видна деградация. Единственный устойчивый сигнал в реальных данных — канал недавно уже отказывал; на нём и построено правило давности. Паспортов, проверок, актов выездов и истории ремонтов заказчик не передаёт и не передаст, поэтому признаков износа оборудования взять неоткуда.
3. **«Неисправен» — потеря связи, а не поломка.** Значительная часть отказов приходит пачками по нескольку каналов разом — это отказ щита, контроллера или шлейфа, а не отдельного датчика, и по истории самого датчика его не предсказать.

Мы исследовали и более сложные модели. Логистическая регрессия по признакам журнала (давность, число отказов за 7, 30 и 90 суток, отказы соседей на пикете и коллекторе, вид датчика) прироста к правилу давности не дала, поэтому в продукт не вошла; её обучение — `ml-model/sensor/train_sensor_model.py`. Модель градиентного бустинга по коллектору целиком (`ml-model/models/lgbm-v3-bag-2026.09.21/`) прогнозирует другую цель — отказ где-либо на коллекторе за 30 суток — и в прогноз диспетчера не входит.

4.5. Производительность

Требование	Что замерено	Результат
Время формирования прогноза меньше 5 минут	весь расчёт по парку: 11 485 датчиков, 3 173 участка, полная база	около 3 секунд; журнал <code>worker</code> на стенде 28.09.2026, прогон 2831: «стадии 6/6, суммарно 3.2 с»
Обновление данных на экране не реже раза в минуту	интерфейс перечитывает данные раз в минуту, уведомление о заявке высокого риска приходит потоком <code>GET /api/alerts/stream</code> с проверкой раз в 5 секунд	расчёт идёт каждые 4 минуты, время последнего пересчёта отдаёт <code>GET /api/data-status</code> в поле <code>computed_at</code>
Не меньше 20 одновременных пользователей	сценарий <code>deploy/load/20-users.mjs</code> : 20 сессий диспетчера по 5 минут, действие раз в 15 секунд, против стенда 28.09.2026	861 запрос к API, ошибок 0; 80-й перцентиль ответа 623 мс, 95-й — 1 237 мс; главный экран в худшей сессии открылся за 2,6 с при пороге 3 с
TLS 1.2 и выше	проверка стенда <code>openssl s_client</code> 28.09.2026	TLS 1.2 и 1.3 принимаются, TLS 1.0 и 1.1 сервер отклоняет с <code>alert protocol version</code> ; для своего стенда протокол снимает <code>deploy/check-tls.sh</code>
PostgreSQL 12 и выше	версия сервера базы	PostgreSQL 18 с PostGIS 3.6; минимальная версия для нашей схемы — 15 (разд. 5)

В нагрузочном замере самый медленный метод — журнал прогнозов `GET /api/forecasts`: медиана 3,1 с. На остальные методы это не влияет, но экран журнала при 20 пользователях открывается заметно дольше других.

4.6. Как повторить замер

Всё, что нужно, лежит в каталоге `ml-model/sensor/` (описание — его `README.md`).

1. Поднять стенд и залить выгрузку заказчика (разд. 9).
2. Выгрузить из базы шесть CSV, около минуты:

```
bash ml-model/sensor/export_data.sh
```

Ожидаемые размеры: `channels` 12 636 строк (каналы справочника и каналы, найденные только в журнале), `object_tree` 95, `failures` 12 139, `ppr_windows` 26, `passports` 11 485, `checks` 21 417.

3. Пересчитать правила и метрики, около полутора минут:

```
uv run --no-project --with numpy==2.3.3 --with pandas==2.3.3 --with scikit-learn==1.7.2 \
python3 ml-model/sensor/rules.py
```

Скрипт перезаписывает `backend/app/domain/sensor_rules.json` и кладёт все числа в `ml-model/sensor/rules.json`. Метрики таблицы разд. 4.2 — ключи `test.real_recency_high` и `test.synthetic_mode_high_on_all`. На наших данных `git diff` после прогона пуст: правила повторяются байт в байт.

4. Сравнение горизонтов — `ml-model/sensor/horizons.py` в том же окружении, числа пишутся в `ml-model/sensor/horizons.json`.

5. Условия и ограничения

Здесь собрано всё, что решение не делает или делает с оговоркой, и причина каждого пункта. Полный перечень пунктов ТЗ с отметкой о выполнении — `tz-requirements.md`.

5.1. Ограничения данных

Ограничение	Причина	Следствие
Прогнозируется потеря связи с каналом, а не поломка датчика	журнал пишет обрыв линии и поломку одним значением; актов выездов не будет	Precision и Recall ниже плановых (разд. 4.4)
Паспорта оборудования, поверки и предупреждения приборов синтетические	реестра оборудования, истории ремонтов и поверок заказчик не передаёт: «не переданы и предоставлены не будут, так как организаторы ими не располагают»	синтетическую часть экран помечает плашкой «Демо: паспорта синтетические» и подписью «симуляция» у причины; режим «без синтетики» включается переключателем или <code>synthetic=0</code>
Прогнозов пожара, подтопления и несанкционированного доступа как отдельных классов нет	разметки пожаров, подтоплений и проникновений в выгрузке нет, графика огневых работ нет; частота включения насосов и прогноз погоды в расчёт не входят	у прогноза один класс — «отказ датчика»; датчики пожарные, газовые, затопления и контроля доступа прогнозируются как все остальные — по риску потери связи
Нет координат	выгрузка обезличена	схема коллектора — линия с пикетами; геометрия для GeoJSON и WKT построена нами (осевые линии коллекторов, участки по 10 м вдоль пикетов, <code>backend/app/ingest/synthetic_geometry.py</code>), ответ API помечает её <code>"geometry_source": "synthetic"</code>
Данные кончатся 30.06.2026	выгрузка окончательная	на стенде включено проигрывание архива: срез расчёта идёт по июню 2026 года вперёд, чтобы появлялись новые предупреждения и заявки, и останавливается на краю данных
Окна плановых работ известны только для двух объектов в 2026 году	график ППР обезличен, ключа к объектам выгрузки нет (разд. 3.5)	остальные плановые демонтажи прогноз принимает за отказы
Критерий прогнозируемости <code>k</code> не посчитан	для него нужна наработка до отказа по типу оборудования, а реестра нет	на приёмке мы говорим «не посчитан»
2021 год исключён	рекомендация заказчика	обучающее окно правил — с 01.04.2022

Ограничение	Причина	Следствие
В выгрузке 11 485 датчиков, а у заказчика около 108 500 (около 100 000 датчиков ОПС и ДУ и 8 500 метановых, доклад 17.09.2026)	выгрузка — десятая часть парка	время расчёта мерено на нашем объёме; расчёт линейен по числу каналов, при десятикратном парке ожидаем десятки секунд при нормативе 5 минут, но не мерили

5.2. Ограничения интеграции

Заказчик предупредил: «Никакие реальные интеграции не предоставляются» и предложил «симулировать любые API». Поэтому внешние системы в поставке заменены эмуляторами.

Внешняя система	Что в поставке	Как подключить настоящую
Поток СМВУ	служба <code>emulator-smvu</code> раз в минуту шлёт показания архива со сдвигом на 364 дня в <code>POST /api/ingest/readings</code>	тот же метод с токеном <code>INGEST_TOKEN</code> . Принятые показания видны в журнале событий, но в прогноз не идут: расчёт стоит на свёртке выгрузки. Задержку «показание — прогноз» не замеряли. Подписчика очереди сообщений (MQTT) нет
Погода	эмулятор Open-Meteo на <code>api</code> , <code>GET /emu/open-meteo/v1/archive</code>	переменная <code>WEATHER_URL=https://archive-api.open-meteo.com/v1/archive</code> , код тот же
Система заявок	эмулятор хелпдеска, <code>GET /emu/helpdesk/v1/tickets</code>	переменная <code>ORDER_SYSTEM_URL</code>
Каталог пользователей	демо-каталог OpenLDAP, служба <code>ldap</code>	три переменные <code>LDAP_URI</code> , <code>LDAP_BASE_DN</code> , <code>LDAP_USER_TEMPLATE</code>
Реестр нарядов-допусков	свои наряды, <code>POST /api/permits</code>	интеграции нет

Во внешние системы сервис ничего не пишет. Загрузки истории из XLSX нет: вся выгрузка заказчика пришла в CSV. Модуля автоматического дообучения нет: правила пересчитываются скриптом `ml-model/sensor/rules.py` вручную (разд. 4.6).

5.3. Технические условия

- **PostgreSQL 15 и выше.** Поставка поднимает свою базу PostgreSQL 18 в контейнере. На своей базе схема накатится только начиная с 15-й версии: в `db/migrations/003_permits.sql` стоит `UNIQUE NULLS NOT DISTINCT`, эта конструкция появилась в PostgreSQL 15. Требование ТЗ «PostgreSQL 12 и выше» наш сервер 18-й версии выполняет.
- **Железо.** Стенд работает на 6 ядрах, 11 ГБ памяти и 119 ГБ диска под Ubuntu 24.04. База с полной историей весит около 55 ГБ; пик места приходится на заливку: около 50 ГБ базы плюс около 22 ГБ журнала упреждающей записи.
- **Настройки PostgreSQL под память** в репозиторий не входят, их задают командой `ALTER SYSTEM` (разд. 9.4).
- **Образ базы собран только под x86_64** (`postgis/postgis:18-3.6`); на Mac с Apple Silicon он работает в эмуляции.
- **Выгрузки заказчика в репозитории нет** (13 ГБ). Без неё сервис поднимается, но экраны пусты: прогнозировать нечего.

- **Персональные данные.** Выгрузка обезличена; единственное поле с персональными данными — ФИО пользователя сервиса (`ref.app_user.full_name`). Разбора соответствия 149-ФЗ, 152-ФЗ и стандартам комплекса ГОСТ 34 мы не делали.
- **Браузеры.** Интерфейс проверен в Chrome (E2E-тесты Playwright, `frontend/e2e/`), в Яндекс.Браузере не проверялся. Отдельных ролей руководителя и аналитика нет: ролей четыре — диспетчер ОДС, диспетчер района, техник, администратор.
- **Резервные копии.** Служба `backup` раз в сутки снимает копию кластера, база между копиями не реже раза в 15 минут кладёт архив журнала. Восстановление `deploy/backup.sh --restore` проверено в изолированной среде: на базе 2,3 ГБ копия заняла 23 с, восстановление — 34 с. На стенде с полной базой время восстановления не замеряли. Копии лежат на том же диске, что и база; выносить их на отдельное хранилище — задача эксплуатации.

6. Функциональная архитектура

6.1. Роли и функции

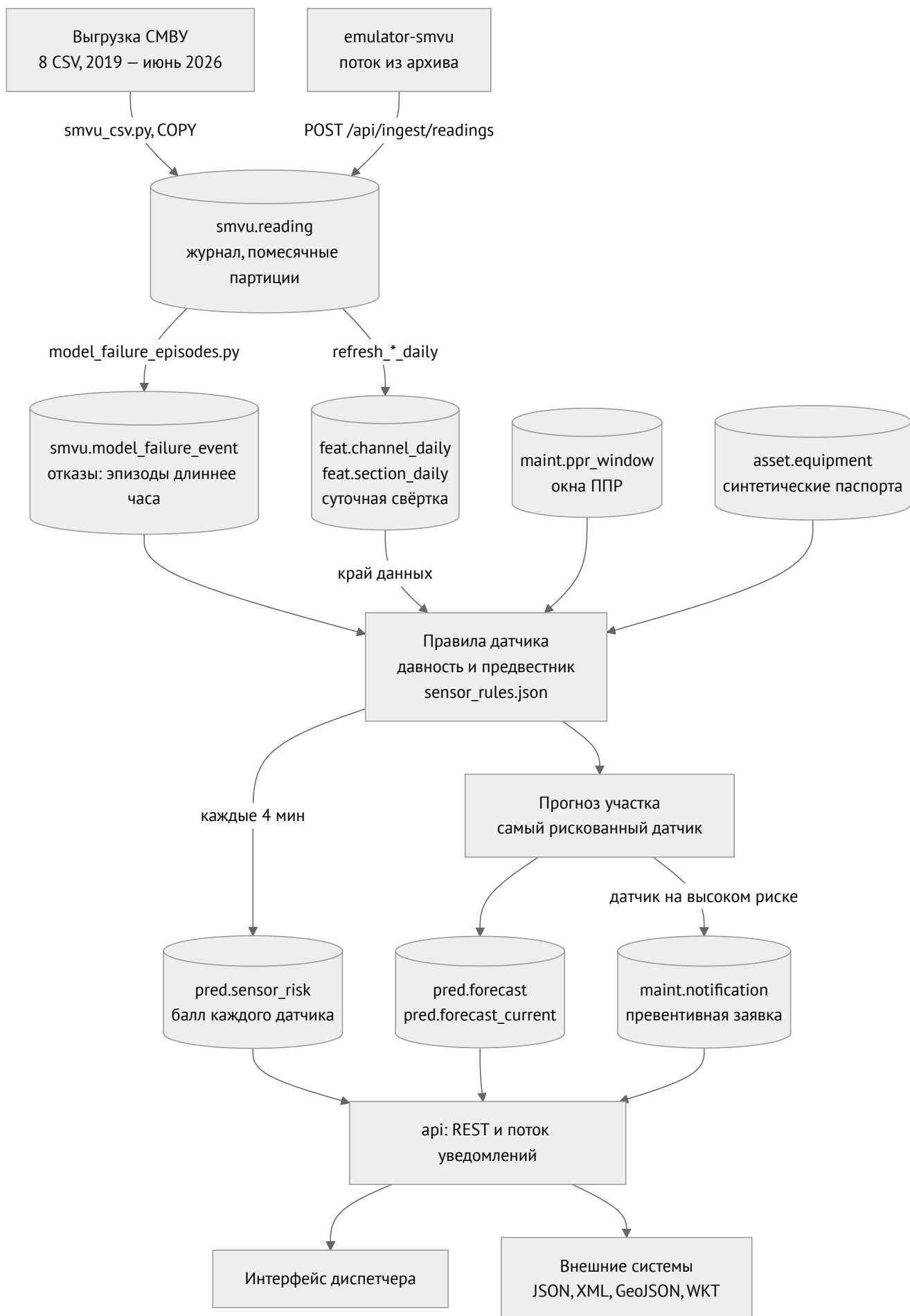
Роль	Что видит и делает
Диспетчер ОДС (ods1)	риски, прогнозы, объекты и заявки по всему парку; решение по прогнозу, квитирование уведомлений
Диспетчер района (disp2)	то же в своём районе – коллекторы «объект Альфа» и «объект Гамма»
Техник (tech1)	риски, прогнозы, объекты и заявки своего комплекса; история отказов канала перед выездом
Администратор ИС (admin1)	пользователи и их блокировка, каталог LDAP, настройки расчёта, состояние источников данных, журнал действий пользователей

Пароли демо-учёток показаны на экране входа. Каждое действие пользователя пишется в журнал `audit.user_action`.

Функция	Экран	Метод API
Сводка рисков по датчикам и коллекторам	«Дашборд рисков», <code>/dashboard</code>	<code>GET /api/sensor-risk/summary</code> , <code>GET /api/sensor-risk</code>
Схема коллектора по пикетам с масштабом до одного пикета, фильтры по риску и типу объекта	«Карта объектов», <code>/map</code>	<code>GET /api/objects/tree</code> , <code>GET /api/risks</code> , <code>GET /api/geo/sections</code>
Карточка участка: датчики, показания, отказы каналов, блок «Почему такой риск»	<code>/objects/<участок></code>	<code>GET /api/objects/{section_id}</code> и вложенные
Журнал прогнозов с моментом расчёта, вероятностью и решением диспетчера	«Журнал прогнозов», <code>/log</code> , <code>/forecasts/<id></code>	<code>GET /api/forecasts</code> , <code>POST /api/forecasts/{id}/feedback</code> , <code>POST /api/forecasts/{id}/outcome</code>
Превентивные заявки со сроком, приоритетом и обоснованием	«Заявки», <code>/orders</code>	<code>GET /api/orders</code>
Журнал технологических событий СМВУ с фильтрами	«Журнал событий», <code>/tech-events</code>	<code>GET /api/tech-events</code>
Уведомления о новых заявках высокого риска без перезагрузки страницы	полоса уведомлений над экраном, вкладка «Неквитированные» в заявках	<code>GET /api/alerts/stream</code> , <code>GET /api/notifications</code>
Администрирование	<code>/admin/users</code> , <code>/admin/directory</code> , <code>/admin/settings</code> , <code>/admin/sources</code> , <code>/admin/audit</code>	<code>GET /api/auth/users</code> , <code>GET /api/settings</code> , <code>GET /api/sources</code> , <code>GET /api/audit</code>

Кнопка «Тур по системе» в интерфейсе проводит по экранам с пояснениями.

6.2. Путь данных от журнала до экрана и заявки



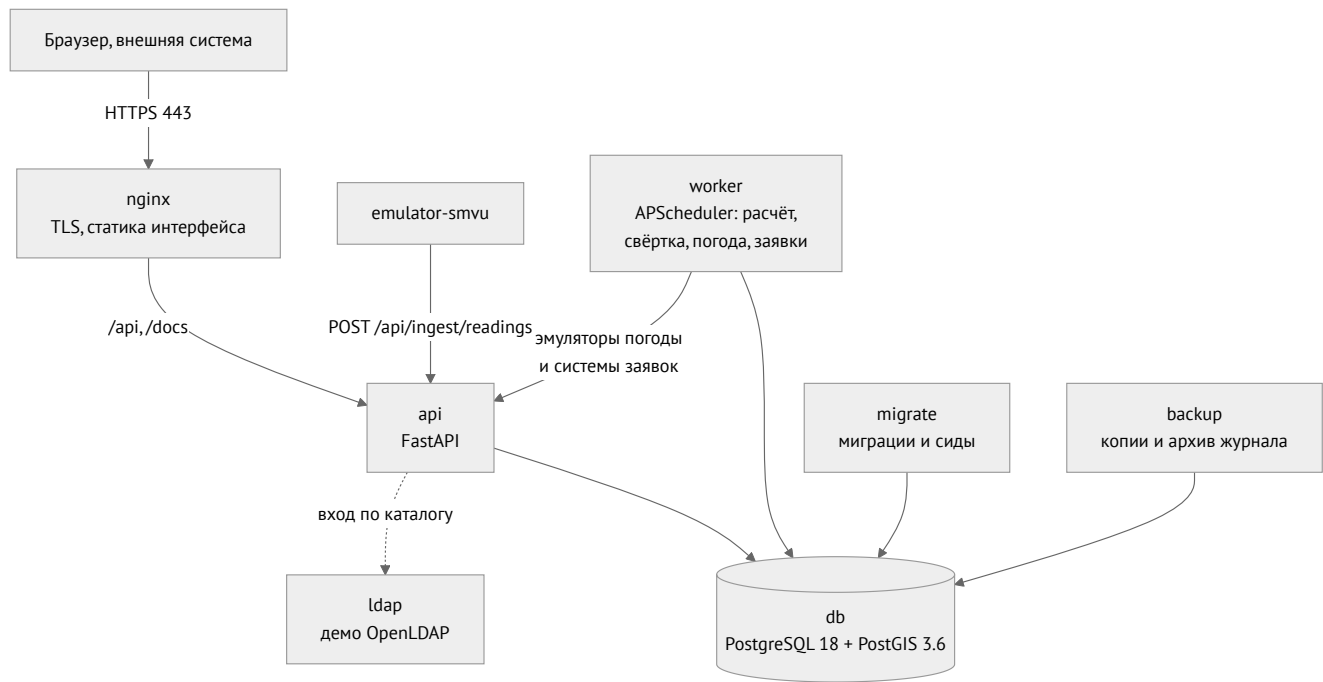
Шаги по порядку.

1. Выгрузка заливается в `smvu.reading` один раз при установке (разд. 3.2). Служба `emulator-smvu` дописывает туда поток из архива, но в прогноз он не идёт (разд. 5.2).
2. Из журнала строятся эпизоды отказа (разд. 3.4) и суточная свёртка (разд. 3.6).
3. Каждые 4 минуты `worker` применяет правила к каждому из 11 485 датчиков на срезе расчёта, исключая окна ППР (разд. 3.5 и 3.7), и пишет балл, уровень и причины в `pred.sensor_risk`.
4. Датчики сворачиваются в участки (разд. 3.8); прогноз ложится в `pred.forecast` — журнал всех прогнозов — и `pred.forecast_current` — текущий прогноз по участку.
5. На участки с датчиком на высоком риске расчёт заводит заявку (разд. 3.9).
6. `api` раз в 5 секунд проверяет новые уведомления — это заявки, заведённые расчётом, — и передаёт их в открытые окна браузера потоком `GET /api/alerts/stream`. Диспетчер видит новый высокий риск без перезагрузки страницы; остальные данные экран перечитывает раз в минуту.
7. Диспетчер отмечает решение по прогнозу, а позже — исход: подтвердилось или ложная тревога с причиной. Эти отметки ложатся в `pred.feedback` и `pred.forecast_outcome`.

7. Компонентная архитектура

7.1. Контейнеры

Система описана одним файлом `deploy/docker-compose.yml` и состоит из восьми контейнеров. Сервисы приложения включены профилем `app`, демо-каталог LDAP – профилем `ldap`.



Контейнер	Образ	Что делает	Связи
db	postgis/postgis:18-3.6	база: журнал, прогнозы, заявки, справочники; архивирует журнал упреждающей записи не реже раза в 15 минут	порт 5432 только на петле хоста
nginx	nginx:1.31-alpine	TLS 1.2 и 1.3, редирект с HTTP на HTTPS, отдаёт собранный интерфейс и файл схемы участков, проксирует <code>/api</code> и <code>/docs</code> в <code>api</code>	порты 80 и 443 наружу
api	собирается из <code>backend/Dockerfile</code>	REST API, вход и роли, поток уведомлений, эмуляторы погоды и системы заявок, приём потока CMBY	к <code>db</code> , к <code>ldap</code> при входе через каталог
worker	тот же образ, команда <code>python -m app.worker.scheduler</code>	расписание из разд. 3.10: расчёт прогноза и заявок, свёртка, погода, статусы заявок	к <code>db</code> , к эмуляторам на <code>api</code>
migrate	тот же образ, команда <code>python -m app.migrate</code>	накатывает <code>db/migrations/*.sql</code> по возрастанию номера, каждый файл в своей транзакции, затем сиды <code>db/seed/*.sql</code> ; живёт секунды и выходит	<code>api</code> и <code>worker</code> ждут его успешного завершения
emulator-smvu	тот же образ	раз в минуту шлёт показания архива со сдвигом на 364 дня, чтобы совпал день недели	к <code>api</code>
backup	postgis/postgis:18-3.6	раз в сутки копия кластера <code>pg_basebackup</code> , хранение последних копий вместе с архивом журнала, восстановление	к <code>db</code> , том <code>backups</code>

Контейнер	Образ	Что делает	Связи
ldap	собирается из deploy/ldap/Dockerfile	демонстрационный каталог OpenLDAP с учётками четырёх ролей	внутри сети compose

Интерфейс — не отдельный контейнер, а статика: сборка frontend кладётся в каталог deploy/nginx/app, который монтирует nginx. Тома: pgdata — база, backups — копии и архив журнала, parquet, uploads и score — рабочие каталоги worker.

7.2. Схема базы данных

Одна база, 11 прикладных схем, 55 файлов миграций с номерами от 001 до 062. Журнал накатанных миграций — public.schema_migration.

Схема	Назначение	Главные таблицы
smvu	журнал и справочники СМВУ, эпизоды отказа	reading (помесечные партии), channel, object_tree, model_failure_episode, fault_episode, data_outage, sensor_kind, system_kind
feat	суточные свёртки	channel_daily, section_daily, refresh_run
pred	прогнозы и решения по ним	run, sensor_risk, forecast, forecast_current, feedback, forecast_outcome
maint	заявки и работы ТОиР	notification, notification_status_log, work_order, ppr_window, ods_event, plan, strategy
asset	техместа и оборудование	func_location, equipment, measuring_point, measurement
ref	справочники и настройки	object_xref (участки), app_setting, app_user, user_role, user_scope, role_permission, priority, criticality
geo	синтетическая геометрия	geo_object
permit	наряды-допуски	permit, permit_status_log, location
load	пакеты заливки и отчёт о качестве	batch, error
audit	журнал действий пользователей	user_action
ext	внешние данные	weather_hourly

Структура asset и maint повторяет модель данных ТОиР: техническое место, единица оборудования, точка измерения, сообщение, заказ. Поэтому настоящий реестр оборудования, если он появится, ляжет в готовые таблицы без изменения схемы.

8. REST API

Полное описание всех 45 методов с параметрами и схемами ответов – Swagger на стенде:

<https://moskollektor.mbogatyreva.ru/docs>. Ниже – основные методы.

Метод и путь	Что отдаёт
POST /api/auth/login	вход по логину и паролю, ставит cookie сессии <code>mk_session</code>
GET /api/auth/me	текущий пользователь и его роли
GET /api/risks	текущий риск по каждому из 3 173 участков: вероятность за 24 ч, класс, ранг
GET /api/data-status	край данных, срез текущего прогноза, время последнего пересчёта
GET /api/sensor-risk	балл, уровень и причины каждого датчика; фильтры по коллектору, узлу, участку, уровню; <code>synthetic=0</code> – без синтетики
GET /api/sensor-risk/summary	сколько датчиков на каждом уровне и коллекторы с наибольшим числом датчиков высокого риска
GET /api/forecasts, GET /api/forecasts/{id}	журнал прогнозов за период и один прогноз с объяснением
POST /api/forecasts/{id}/feedback, POST /api/forecasts/{id}/outcome	решение диспетчера и исход прогноза
GET /api/orders, GET /api/orders/{id}	превентивные заявки со сроком, приоритетом и обоснованием
GET /api/objects/tree, GET /api/objects/{section_id}	дерево «коллектор – узел» и карточка участка
GET /api/objects/{section_id}/readings, .../channels, .../channels/{channel_id}/episodes	показания, каналы и история отказов канала
GET /api/geo/sections	геометрия участков в GeoJSON или WKT
GET /api/tech-events	журнал технологических событий СМБУ с фильтрами
GET /api/notifications, POST /api/notifications/{id}/ack	уведомления и квитирование
GET /api/alerts/stream	поток уведомлений о новых заявках (Server-Sent Events)
POST /api/ingest/readings	приём показаний СМБУ пачками, авторизация токеном
GET /health	живость сервера без обращения к базе

Аутентификация. Вход – POST /api/auth/login с телом `{"login": "ods1", "password": "..."};` сервер ставит cookie `mk_session` и дальше узнаёт пользователя по ней. Учётки бывают локальные (пароль хранится хешем `argon2`) и из каталога LDAP. Вход только по паролю или через каталог LDAP. Без сессии методы отвечают 401, без нужного права – 403 с названием права. Каждая роль видит только свою область: диспетчер района – свои коллекторы, техник – свой комплекс. Приём потока POST /api/ingest/readings авторизуется заголовком `Authorization` с токеном `INGEST_TOKEN`; пока токен не задан, метод отвечает 503.

Форматы. По умолчанию все методы отвечают JSON. XML отдают все методы, отвечающие JSON, — по параметру `?format=xml` или заголовку `Accept: application/xml`. Геометрию участков `GET /api/geo/sections` отдаёт в GeoJSON (`?geometry=geojson`, тип `application/geo+json`) или WKT (`?geometry=wkt`); координаты в системе EPSG:4326, поле `geometry_source` у синтетической геометрии равно `synthetic`.

Пример на стенде, 28.09.2026, после входа под `ods1`:

```
GET /api/sensor-risk/summary
{"synthetic": true, "as_of": "2026-06-01T06:00:00+00:00",
 "high": 11, "watch": 3, "normal": 11471, "collectors_with_high": 6, ...}

GET /api/geo/sections?geometry=wkt&section_id=369
{"crs": "EPSG:4326", "geometry_source": "synthetic",
 "items": [{"section_id": 369, "smvu_key": "1051:968",
 "wkt": "LINESTRING(37.633227215 55.842741149,37.633227215 55.842830966)"}]}
```

```
GET /api/data-status?format=xml
<?xml version='1.0' encoding='utf-8'?>
<response><data_edge>2026-06-30T20:59:59+00:00</data_edge>...<sections_total>3173</sections_total>
</response>
```

В первом ответе $11 + 3 + 11\,471 = 11\,485$ — все активные датчики выгрузки.

9. Сборка и установка

Каждый шаг ниже подробно, с ожидаемым выводом, описан в `deploy/README.md` репозитория.

9.1. Что нужно на машине

- Linux на x86_64; проверено на Ubuntu 24.04, 6 ядер, 11 ГБ памяти, 119 ГБ диска. Для полной выгрузки нужно не меньше 75 ГБ свободного места (разд. 5.3).
- Docker с плагином `docker compose` (проверено на Docker 29.6–29.8, Compose 5.3–5.5).
- Node.js для сборки интерфейса (сборка на GitHub Actions идёт на Node 26).
- Python 3 с пакетом `asynpcpg` — для заливки выгрузки с хоста.
- Выгрузка заказчика: два справочника и восемь файлов `ext-journal-*.csv`.

9.2. Поднять систему

Команды выполняются из каталога `deploy` клонированного репозитория.

```
cp .env.example .env
# вписать в .env POSTGRES_PASSWORD (вывод openssl rand -hex 16)
# и AUTH_SECRET (вывод openssl rand -hex 32); без них compose не стартует
sh make-cert.sh # самоподписанный сертификат в nginx/certs
docker compose up -d # db, nginx, backup
docker compose --profile app up -d # миграции, API, расчёт и эмулятор CMBU
cd ../frontend && npm ci && npm run build \
&& mkdir -p ../deploy/nginx/app && cp -R dist/. ../deploy/nginx/app/
```

Интерфейс откроется на <https://localhost> (сертификат самоподписанный, браузер предупредит). Демо-каталог LDAP поднимается отдельно: `docker compose --profile app --profile ldap up -d`.

9.3. Залить выгрузку

Из корня репозитория, пароль — `POSTGRES_PASSWORD` из `deploy/.env`:

```
python3 -m venv .venv && .venv/bin/pip install asynpcpg
DSN="postgresql://moskollektor:<пароль>@127.0.0.1:5432/moskollektor"
PYTHONPATH=backend .venv/bin/python -u -m app.ingest.smvu_csv --dsn "$DSN" \
--objects <каталог>/справочник_объектов_диспетчер.csv \
--channels <каталог>/справочник_каналов_датчиков.csv \
<каталог>/ext-journal-*.csv
PYTHONPATH=backend .venv/bin/python -m app.ingest.smvu_csv --dsn "$DSN" --check
```

В один поток заливка идёт три с лишним часа, в три потока (ключ `--slot`) — около часа; раскладка по потокам и построение индексов после неё (`--build-indexes`) описаны в `deploy/README.md`. Заливку лучше вести на сервере: база вырастает до 55 ГБ.

После заливки:

1. Повторить заливку справочника каналов (`--channels`) — она дозаполнит коллектор и вид места уже залитым каналам и нарисует синтетическую геометрию: в журнале появится строка `геометрия: коллекторов 16, частей 32, участков 3173`.
2. Построить эпизоды отказа внутри контейнера `api`: `docker compose exec api python -m app.ingest.model_failure_episodes`.
3. Заполнить суточные свёртки за всю историю — по ним сервис определяет край данных:

```
docker compose exec -T db psql -U moskollektor -d moskollektor -c \  
"SELECT feat.refresh_channel_daily('2019-01-01', '2026-06-30');\  
SELECT feat.refresh_section_daily('2019-01-01', '2026-06-30');"
```

4. Прогнать migrate ещё раз: `docker compose --profile app run --rm migrate`. При первом накате каналов ещё не было, и сид синтетических паспортов `db/seed/sensor_demo.sql` пропустил их с сообщением `sensor_demo: активных каналов нет, пропускаю`.

9.4. Настройки базы под железо

Для 11 ГБ памяти и 6 ядер задаются командой `ALTER SYSTEM` и перезапуском `db`: `shared_buffers = '3GB'`, `effective_cache_size = '8GB'`, `maintenance_work_mem = '1GB'`, `work_mem = '64MB'`, `max_wal_size = '8GB'`, `checkpoint_timeout = '15min'`, `random_page_cost = '1.1'`, `max_parallel_workers_per_gather = '2'`. На другом железе `shared_buffers` берётся на четверть памяти, `effective_cache_size` — на две трети.

9.5. Убедиться, что всё поднялось

`docker compose up -d` возвращает 0, даже когда служба упала сразу после старта, поэтому проверять надо отдельно.

```
docker compose ps                # db, nginx, api — (healthy)  
docker compose exec -T db pg_isready # accepting connections  
curl -sk -o /dev/null -w '%{http_code}\n' https://127.0.0.1/           # 200  
curl -s -o /dev/null -w '%{http_code}\n' http://127.0.0.1/             # 301  
curl -sk -o /dev/null -w '%{http_code}\n' https://127.0.0.1/api/risks # 401 без входа  
docker compose logs worker | grep "прогон"                            # «стадии 6/6, суммарно ... с»  
docker compose exec api python code/check_schema.py                   # порядок миграций и целостность  
sh check-tls.sh 127.0.0.1                                              # протокол TLS: 1.2 и 1.3, без 1.0 и 1.1
```

Полный прогон приёмочных проверок из распакованного пакета — `bash delivery/check-all.sh`.

9.6. Резервные копии и снос

Служба `backup` поднимается вместе с базой. Копия сейчас — `docker compose exec backup sh /backup.sh`, учения с восстановлением рядом — `... sh /backup.sh --drill`, восстановление после аварии — `sh backup.sh --restore`. `docker compose down` убирает контейнеры и сохраняет данные; `docker compose down -v` стирает базу вместе с копиями.

10. Стек и лицензии

Слой	Технология	Версия
База	PostgreSQL + PostGIS	18 + 3.6, образ <code>postgis/postgis:18-3.6</code>
Бэкенд	Python	3.14, образ <code>python:3.14-slim</code>
	FastAPI, Starlette, Pydantic, Uvicorn	0.141.1, 1.6.0, 2.13.5, 0.52.4
	asyncpg — драйвер PostgreSQL	0.31.0
	APScheduler — планировщик	3.11.3
	argon2-cffi — хеши паролей, python-ldap — вход через каталог	25.1.0, 3.4.8
Веб-сервер	nginx	1.31, образ <code>nginx:1.31-alpine</code>
Интерфейс	Preact, preact-router, driver.js	10.29.8, 4.1.2, 1.8.0
Сборка интерфейса	Vite, TypeScript, Tailwind CSS	8.3.0, 7.0.2, 4.3.3
Тесты интерфейса	Playwright	1.63.0
Расчёт правил (офлайн)	numpy, pandas, scikit-learn	2.3.3, 2.3.3, 1.7.2
Развёртывание	Docker Compose, GitHub Actions	—

Все зависимости бэкенда, включая транзитивные, закреплены точной версией в `backend/requirements.txt` (23 пакета), интерфейса — в `frontend/package-lock.json`.

Интерфейс собран на Preact, а не на React с Next.js: пустая заготовка Next.js весит 107 КБ сжатого JavaScript при нашем бюджете 150 КБ на весь интерфейс. Схема коллектора нарисована на SVG без картографической библиотеки: координат нет, а заказчик написал, что схематичное представление «допустимо и даже рекомендуется, если оно нагляднее (например, линейная схема с пикетами)». Стек совпадает с тем, на котором у заказчика работает действующая CMBU, — PostgreSQL, Python, NGINX, Debian.

Лицензии. В образы `api`, `worker` и `nginx` и в код, который уходит в браузер, не входит ничего под GPL, LGPL и AGPL. Лицензии пакетов бэкенда — MIT, BSD, Apache-2.0, PSF и лицензия `python-ldap` в стиле Python; `nginx` — BSD-2-Clause; в браузер уходят Preact, preact-router и driver.js под MIT. Проверку по метаданным пакетов делает `code/check_licenses.py`. Инструменты сборки в поставку не попадают; среди них один пакет под MPL-2.0 (`lightningcss`, движок CSS у Tailwind), он не из запрещённого списка. Нагрузочный сценарий написан на Node без зависимостей, потому что `k6` распространяется под AGPL-3.0.

Две оговорки. Сервер базы — стандартный образ PostgreSQL с PostGIS: PostgreSQL распространяется под лицензией PostgreSQL, PostGIS — под GPL-2.0. Образ используется без изменений как отдельный сервер, наш код обращается к нему по сетевому протоколу и с ним не компонуется. Демо-каталог `ldap` собран из пакетов Debian (OpenLDAP под OpenLDAP Public License 2.8, среди транзитивных пакетов есть GPL-утилиты); он поднимается отдельным профилем и нужен только для демонстрации входа через каталог.

11. Как проверить решение за 15 минут

1. Открыть <https://moskollektor.mbogatyreva.ru>, на экране входа нажать «Войти как» у диспетчера ОДС ods1. Дашборд покажет, сколько датчиков на высоком риске и на каких коллекторах. Переключатель «Учитывать синтетику: паспорт и предвестники» убирает синтетическую часть и оставляет только правило давности.
2. Открыть «Карту объектов»: схема коллектора по пикетам, датчики с риском отмечены на трассе, колесом мыши схема масштабируется до одного пикета. Щелчок по участку открывает карточку с блоком «Почему такой риск».
3. Открыть «Заявки»: заявки заведены расчётом, у каждой срок, приоритет и обоснование с названием датчика и правила; из заявки открывается прогноз.
4. Открыть <https://moskollektor.mbogatyreva.ru/docs> и вызвать, например, `GET /api/risks` или `GET /api/geo/sections?geometry=wkt`.
5. Повторить метрики по разд. 4.6 на своей копии базы.